

Ajax en formularios

Trabajo con formularios y Ajax

JSON nativo



```
var persona = {  
  nombre: "Miguel",  
  edad: 38  
}
```

```
var cadena = JSON.stringify(persona);  
var obj = JSON.parse(cadena);
```

Ajax con \$.post()



Es la función que usaremos para enviar datos por post.

- Recordar que es un "shortcut" de \$.ajax(), que sirve para todo.
- El funcionamiento es similar a lo visto para \$.get(), pero envía los datos por POST en vez de GET.
- Las cosas nuevas que vamos a conocer hoy las puedes aplicar por tanto a \$.get u otros "shortcuts" de \$.ajax().

<http://api.jquery.com/jQuery.post/>

Ajax con \$.post()



No necesitas tener un formulario para enviar datos por POST. Simplemente llama a \$.post()

```
$.post('pag.php', {  
    dato: "valor",  
    dato2: "valor2"  
}, function(respuesta) {  
    $('#resultado').text  
(respuesta);  
});
```

Serialización de formularios

Los datos de un formulario habría que enviarlos, pero no necesitas componer a mano todo el juego de {data} extraído de cada uno de los campos de formulario.

Utilizas la función de serialización.

<http://api.jquery.com/serialize/>

```
$("#formulario").serialize()
```



jqXHR: el "objeto Ajax"



Ese objeto nos sirve para obtener info. de la solicitud Ajax y realizar acciones con ella.

A partir de jQuery1.5 lo obtienes como retorno de una llamada a \$.post() (u otras funciones Ajax de jQuery)

```
var objAjax = $.post(url, form.serialize(), FunExito);
```

jqXHR: el "objeto Ajax"



```
objAjax.done(function() {  
    //Defino acciones cuando completa con exito  
});  
  
objAjax.fail(function() {  
    //Defino acciones cuando ocurre error  
});  
  
objAjax.always(function() {  
    //Defino acciones tanto en éxito como error  
});
```